30 pillars of YT Cluster

Updated October 2026 | 17 minute read

DBinvent Engineering

From 30 years of personal IT experience into 30 pillars of the solution: what each one does, how it works, and where it is tested in public. The first 3 are the ones the front page leads with, and the next 6 its essential features.

Beyond client-server#

Read as a database - PostgreSQL that writes on every node - YT Cluster is too small a story. Run on your own machines, it is an on-premises micro-cloud. And the point it goes on to is bigger still: it replaces the client-server model with a local-first architecture.

An embedded PostgreSQL node runs *on the client* and is a real, writable member of the cluster, so the client reads and writes locally and the cluster's sync carries every change on - no API call, no request and response, no REST tier for the state it keeps in sync. That is local-first, but transactional and multi-master, which local-first normally is not.

> The client-server model is giving way to local-first, and YT Cluster is the only transactional, real-PostgreSQL way to do it - not just a faster database.

Getting there moves no data: the cluster installs into the PostgreSQL you run today, which may not even need a restart - none where `max_prepared_transactions` is set already - while other systems make you move your data into theirs (into a cluster and back out). There is no vendor lock-in: it is PostgreSQL, unmodified, with all of its features (pillars 2 and 3). And it is AI query ready: ask in plain language, and `yt_llm()` answers in SQL, on your own hardware (pillar 15).

Who is it for?#

Something useful for every company. Everyone who runs PostgreSQL: among these 30 pillars, every team finds something that pays off for it. And the cluster grows as your needs do - more nodes, zones, tiers, linked clusters - with no data migration on your side: it copies and moves its own data while it keeps taking writes. It is the path Schema Guard gives a schema: start with one collaborative YAML file, move on to versioned, signed-off scripts when your process needs them.

Every subsystem's status in the current version - shipping, built, off by default, fixed but not yet green, or not built - is kept in agent.md, and the nightly results are the newest word on all of it.

01. Performance growth without limits
02. No vendor lock-in
03. Same tech stack, simple onboarding
04. High availability, multi-master
05. No cap on nodes
06. Survives network outages
07. Self-healing, verified
08. Consistency per table
09. Topology by rules, changed online
10. Serverless Python, built in
11. Distributed cache and queue
12. KV and point-to-point streams, any language
13. Key to queue to Python - no SQL on the way
14. Secret key-value store
15. LLM-ready

16. Built-in MPP
17. ClickHouse in one click
18. Schema management, without Java
19. Cluster-ready migrations
20. Dynamic schema adaptation, open source
21. Hierarchical and regional boundaries
22. Operated through standard SQL
23. Secret management and security
24. Audit ready
25. Console UI installer
26. Add a node with no data
27. Agent ready
28. Non-blocking concurrent document operations
29. Leave any time - a full rollback
30. A Rust server - no JVM

01 · Performance growth without limits#

> Grow as far as the load goes, with no vendor change and no data transfer - still your favorite, proven PostgreSQL.

In a primary-and-replicas cluster exactly one node may write, so write capacity is the size of that machine. Here every node accepts writes, and each table says how many nodes must confirm its commits, so a commit waits only for what its table needs. Measured with three nodes against a standard three-node PostgreSQL synchronous-replication setup (a primary and two standbys): **1.7× the throughput** on a write-only load and **1.6×** on an 80/20 read/write mix. Full synchronous two-phase commit to every node costs more than a plain synchronous replica, and we say so: that setting is there for the tables that need it.

Past one cluster, clusters link into a hierarchy, each with its own leader and quorum, so growing means adding clusters rather than stretching one consensus group. Linear scaling is a property of the architecture. Absolute TPS figures come from older lab hardware. The 2-to-10-node curve is next.

Tested: the benchmark · Scaling writes

02 · No vendor lock-in#

> Your own solution: like a cloud database, but cheaper and better - your hardware, your rules, nobody else's price list.

Nothing in the stack belongs to a provider: unmodified PostgreSQL, one package from the public apt and yum repositories, and machines you choose - your own racks, a colocation, or plain virtual machines in any cloud. The data sits in ordinary PostgreSQL tables that every PostgreSQL tool reads.

A managed database rents you availability: its primary, its region, its price list. Here the availability is in the software - every node writes, copies are verified, a returning node catches up by itself - on machines you own, and the way out is documented and tested (pillar 29).

Why there is no DBinvent cloud

03 · Same tech stack, simple onboarding#

> No learning curve and no source code change: the same proven PostgreSQL code, SQL, drivers and tools. Your team is onboarded already - and simple onboarding is money saved.

The engine is the PostgreSQL you run today, unmodified: the same SQL, drivers, connection strings, extensions, backups and monitoring. A table joins the cluster by configuration, not by a schema change; there is no new query language, no shard key to retrofit and no proxy to point the applications at.

What your team already knows about PostgreSQL stays true, so onboarding is days of reading rather than a migration project - for the

developers, the DBAs and the operations staff alike.

Getting started

04 · High availability, multi-master#

> Every node writes - strong on write and still fast. A usual PostgreSQL cluster is a complicated manual setup: a single primary with a standby, a limited number of replicas that are slow when strongly consistent, or eventual consistency.

Every node accepts reads and writes. A leader is elected, but it arbitrates order rather than carrying the traffic: conflicts are found at write time, before a transaction commits, and changes then apply in the same order on every node. Two `v = v + 1` updates from the same starting value cannot both commit - the second is refused with SQLSTATE `40001`, the error every driver already retries, and its retry lands on top.

When a node goes away, the others were already taking writes: no promotion step, no failover window, no split-brain drill. A node that comes back replays what it missed, in order, and asks the leader to take part again.

Tested: topology 29 and 31 (concurrency, the commit path); 35 resilience runs, no acknowledged write lost in about 14,300 · Fault tolerance

05 · No cap on nodes#

> Other cloud databases with two-phase-commit transactions limit the number of nodes, and get slower on writes as they grow. Here a write waits only for the nodes its table asks for, and clusters link into a hierarchy.

When every write must wait for every node, each node added makes writing slower. Here a write waits only for the nodes its table asks for -

half the cluster, at least N, or none - and the rest take it from the cluster log, in order. A node holds only the tables and shards its zone's rules place on it.

Past a single cluster, clusters link as upper, lower and neighbour, each with its own leader, quorum and data (pillar 21): the system grows by adding clusters, not by stretching one consensus group.

Tested: rmc: linked clusters growing from 11 to 17 nodes under load · Where rows live

06 · Survives network outages#

> A node cut off by the network keeps running and writing the part of the schema you allow, and catches up when the link is back.

A site that must keep working alone - a store, a plant, a vessel - runs a cluster of its own, even of one node, linked to the level above. When the link drops, the site keeps reading and writing everything it holds; when the link returns, the tables a rule sends up are relayed from the cluster log and catch up on their own.

A rule per table decides what crosses the link at all: alarms and totals go up, raw readings stay on site. Inside one cluster, a table that needs a majority does not commit a write it cannot get confirmed, rather than let the copies diverge.

Tested: hier (three linked clusters), rmc (the relay) · Disconnected operations

07 · Self-healing, verified#

> Copies are re-checked in the background and repaired by majority vote. A row nobody can vouch for is blocked, not guessed at.

Rows already replicated are re-read in the background, checksummed on every node that holds them, and voted on. Unanimous, and they are done. A strict majority, and the nodes outside it pull the winning copy, overwrite their own and queue it to be confirmed again. Every node runs its own rounds for its own copy, because a leader-only scan could only ever repair the leader.

No majority, or a row still divergent after repeated rounds, and the row is blocked cluster-wide and reported in `yt_alert`. Placement is verified the same way: a partition found on the wrong node for the rules in force is moved online. A write a node lacks altogether is found by comparing its tables with the members, and filled from them.

Tested: topology 80, 88, 89 and 89b; blocking 85-87 · How repair works

08 · Consistency per table#

> Each table sets how many nodes must confirm a commit - half, N, or none - and whether conflicting writes are refused. One cluster holds both the strict tables and the fast ones.

Two settings per table. How many nodes must confirm a commit: `half` the cluster, `nodes>=N`, or asynchronous - the write does not return until two-phase commit has it durable on that many. And how conflicts are treated: by default ordering between *different* rows is tolerated; with sequential consistency a conflicting commit is refused with `40001` and retried.

At every level, an update of a keyed row is admitted only when the writer's copy holds every earlier write of that row, so two increments from the same base cannot both commit. Document merge (pillar 28) and parallel apply are per-table choices too: a ledger and a session cache share one cluster, each with the guarantees it needs.

Tested: topology 29 and 31 · How scope is configured

09 · Topology by rules, changed online#

> Zones, tiers, shards and data residency are declared as rules. When a rule changes, partitions move while being written to - no redeploy, no downtime.

Zones, tiers, shards and data residency are rules in the configuration table, not a deployed shape. A change is a row of the `yt_job` table, approved from the level above and locked per table: create or delete a zone, move a shard or a table's zone, clean up stale copies, bring a table that lives on one node under sync, split a zone off as a cluster of its own, or combine it back.

While a move runs, the old rule stays in force until the targets have filled, then flips, so the table is written to the whole time. And because the topology is declared, a query planner can plan against it - which is what makes the built-in MPP possible (pillar 16).

Tested: topo t0-t7; topology 23, 30 and 95 · Software-defined topology

10 · Serverless Python, built in#

> A serverless framework like AWS Lambda, inside the cluster: Python functions run on insert, update and delete, next to the data.

Python functions triggered by insert, update and delete, registered in the configuration table and run by the node that sees the change - the shape of AWS Lambda or Azure Functions, next to the data, with no event bus to operate. Inside a function, `DB`, `ETCD` and, where ClickHouse is on, `CH` are bound for you.

A function that keeps failing is held by a circuit breaker instead of hammered, its events waiting until a probe succeeds; an adaptive concurrency limit with a rate cap keeps a burst from swamping it; and a schedule can watch a function's queue and send a message when work is due or overdue.

Tested: topology 92, 92b, 92c and 92d; pipeline · Server functions

11 · Distributed cache and queue#

> A built-in distributed key-value cache with a message queue, behind a standard etcd API - no extra cluster to run.

Every node can serve a standard etcd API v3 key-value store, replicated to its peers with quorum counted over the online ones; keys can be confined to a zone. On top of it sits a named queue with ordering and delivery guarantees: a point-to-point mailbox, where each message is bound to exactly one consumer until that consumer acknowledges it.

It is built in, so there is no separate cache or broker to run beside the database. Two things to know: nothing reaps a queue, so use one queue per client rather than one per request; and between nodes a message travels as one call today - a persistent stream between peers is the next phase.

Tested: topology 93 · etcd interface

12 · KV and point-to-point streams, any language#

> Key-value, change streams and point-to-point queues all speak the standard etcd API v3, so any language with an etcd client can use them - no vendor API to learn.

The key-value store, the change streams and the point-to-point queues all speak the standard etcd API v3, so any language with an etcd client - Go, Python, Java, JavaScript, Rust, C# and more - works with no vendor SDK. `Watch` is bidirectional: subscribe to a key range and receive changes as they happen, with no second system for streaming.

Service discovery, configuration and leader election in your own services can use the same surface. The server is open source.

Tested: topology 93 · etcd server

13 · Key to queue to Python - no SQL on the way#

> A message written to a queue reaches your Python function with no SQL and no database on the way, so none of their latency. When the function needs to save or load, the `DB` binding is right there.

A message put on a queue goes straight to the Python function that consumes it, over the etcd interface: no SQL, no table and no transaction on the way, so none of their latency or locking. When the function does need the database, the `DB` binding is there to save or load.

The same path can skip Python entirely: a queue named `/q/ch:<table>/` is read by the etcd server itself and written into that ClickHouse table in batches, with no watcher and no PostgreSQL in between.

Tested: pipeline · Server functions

14 · Secret key-value store#

> Built in: secrets kept encrypted in the cluster itself, readable only on the nodes you approve - no separate vault to deploy.

Secrets live in the configuration table, each value sealed under a master key that is wrapped for every node an operator approved, so only approved nodes can open them. There is nothing else to deploy, and a listing shows names and versions, never values.

The master key is rotated while the cluster runs; a retired node's wraps are deleted and its key is listed as revoked; an optional recovery key restores access if every approved node lost its own.

Tested: topology 34, 35 and 36; rmc; hier · Platform capabilities

15 · LLM-ready#

> Natural language to SQL, available today. No GPU on every node - one model box on the side serves the whole cluster.

`select * from yt_llm('…')` turns a question into SQL with a local model and runs it on the engine that holds the whole answer: PostgreSQL, DataFusion across the nodes, or ClickHouse. `yt_llm_query()` returns the statement instead, for review. A question no engine here can answer whole is refused, with the reason, rather than answered from part of the data.

The models run on CPU, on one host of your own: no GPU in every node, and no question or data sent to an outside service.

Tested: llm: 101 of 101 · yt_llm()

16 · Built-in MPP#

> Massively parallel analytics on Apache DataFusion - up to 4× faster than Spark - across the nodes you already run.

Analytical queries are planned and run by Apache DataFusion across the declared topology, over the tables already in the cluster: nothing is copied into a warehouse first. Because rows live where the rules say, the planner knows where to send each part - `yt_info('p:<table>/<n>')` gives a table's layout in `n` partitions that cover it exactly once.

For many analytics that removes the second system altogether; for columnar workloads there is ClickHouse, next.

Tested: topology 95, 97 and 98 · MPP analytics

17 · ClickHouse in one click#

> One command sets up ClickHouse as a shadow cluster for your analytics data.

`ytctl --clickhouse` installs and configures ClickHouse, its coordination included, as a shadow cluster for the analytics data. PostgreSQL stays the transactional engine and ClickHouse the columnar one; neither is asked to be the other.

Data reaches it from the tables, from server functions through the `CH` binding, or from a queue that the etcd server writes straight into a ClickHouse table in batches.

Tested: topology 94; pipeline · ClickHouse integration

18 · Schema management, without Java#

> Regular schema management, as with Flyway: versioned, immutable and repeatable SQL scripts, tracked by checksum per environment, with full `psql` support and built-in ETL - in a single binary, with no JVM and no JDBC.

Schema Guard does what Flyway does - versioned, immutable and repeatable SQL scripts, tracked by checksum per environment, with full `psql` support - and adds a whole schema declared in one YAML file and turned into SQL for you, mixed with scripts as you like, and built-in ETL. It is a single binary: no JVM and no JDBC driver on the build agents or the servers.

It is a package of its own, `apt install schema-guard`, from the same repository as the cluster.

Schema Guard

19 · Cluster-ready migrations#

> Unique to Schema Guard: each migration is ordered against ordinary writes on every node of the cluster - otherwise the nodes stop agreeing about what their data means. ytserv applies its migrations with Schema Guard's library, and the tool itself is `apt install schema-guard`.

On a cluster, a migration cannot simply run on each node in turn: while it rolls out, the nodes would disagree about what their data means. Schema Guard orders each migration against the ordinary writes on every node, as one more change in the cluster's order.

A schema gate watches the result: each node compares its live schema with the leader's, and a table whose schemas have drifted is held out of row comparison until they converge, instead of flooding the repair with false differences.

Tested: topology 32 · Schema migration

20 · Dynamic schema adaptation, open source#

> It reads the schema an existing database already has, and applies only the declarative changes needed to match the one declared in YAML. This part is open source, MIT licensed.

Point it at an existing database and the schema it should have, declared in YAML: it reads the live schema, works out the difference, and applies only the changes needed to match. No hand-written `ALTER` per environment, and no guessing which scripts an old database has already seen.

This engine is open source under the MIT licence, as schema_guard on GitHub and `schema_guard_tokio` on crates.io.

schema_guard_tokio

21 · Hierarchical and regional boundaries#

> Link clusters vertically and side by side. Up the hierarchy go the analytics, down it come the domain dictionaries - only the tables a rule names, as read-only copies. Regions are neighbours: each cluster keeps its own leader, quorum and data.

Clusters link as upper, lower and neighbour. Each keeps its own leader, quorum and data, and nothing of one cluster's consensus reaches another. The only data that crosses a link is the relay: a read-only copy of the tables a rule names, carried from the cluster log.

Up the hierarchy go the analytics and the totals; down it come the domain dictionaries - price lists, catalogues, reference data. Regions sit side by side as neighbours, each under its own quorum. A lower cluster's jobs are approved by its direct upper, never by one further up, and linked clusters admit each other by their certificate authorities.

Tested: hier (three linked clusters); rmc · Three linked clusters, tested

22 · Operated through standard SQL#

> Almost every operation is plain SQL: settings and operator actions are rows in one configuration table, jobs are rows too, and status is a `select` - no custom command set to learn.

Settings are rows of one configuration table, and they replicate like any other write. Operator actions are rows too: pausing or resuming a node is a request in its `node_run` row, which lets the leader refuse one that would take the cluster below quorum. Topology jobs are rows of `yt_job`,

status is `select * from yt_info('E')`, and alerts land in `yt_alert`.

So the tools you already use - `psql`, scripts, change review - operate the cluster too, with no custom command set or console to learn.

Tested: topology 24 and 25 (the configuration tables) · Configuration table

23 · Secret management and security#

> A node joins only once an operator approves it, and nodes talk over mutual TLS with certificates from the cluster's own CA. The master key is wrapped per approved node, rotated online, revoked with a retired node and recoverable with a key you keep.

A node joins only once an operator approves it. With `peer_tls` on, the cluster runs its own certificate authority: every node gets a certificate, and TLS between members is required once every node holds one. A node that joins later trusts the cluster's certificates only through the CA fingerprint it was given at install.

The master key behind the secret store is wrapped per approved node, rotated online, revoked with a retired node and recoverable with a key you keep. Linked clusters admit each other by their CAs, countersigned from above.

Tested: topology 34, 35, 36 and 98_skv_join; rmc; hier · Platform capabilities

24 · Audit ready#

> The source code of `ytctl`, `ytserv` and the PostgreSQL extension is open to audit at any time, under either license:

> a security review or a compliance check gets the code itself, not a promise.

The source code of `ytctl`, `ytserv` and the PostgreSQL extension is available for audit on request, under either license: a security review or a compliance check reads the code itself, not a datasheet. The parts around it are open source: the etcd server, rppd, diff-doc-rs and Schema Guard's engine.

The evidence is public too: every end-to-end suite and every nightly result, failures included, in the public test repository.

Source available for audit

25 · Console UI installer#

> `ytctl` walks you through installing a node in a console UI: it checks the dependencies and the PostgreSQL version, then installs the extension, the configuration table and the service, showing every command it runs. A second console UI controls the running cluster, and `--batch` installs unattended.

`ytctl` installs a node through a console UI: it checks the dependencies and the PostgreSQL version, then installs the extension, the configuration table and the service, showing every command as it runs it. A second console UI controls the running cluster.

`--batch` installs unattended, for automation; `--clickhouse`, `--skv-*` and `--link-*` set up ClickHouse, the secret store and links between clusters. The installed package is tested end to end: install, unattended install, purge, and the failure cases.

Tested: lxc · ytctl

26 · Add a node with no data#

> A new node starts empty, copies the tables it holds from a member while the cluster keeps writing, and joins once it has caught up. Pausing and rejoining are requests the leader approves.

A new node starts empty. It copies every synced table it holds from a member while the cluster keeps writing, replays the overlap, and only then asks to follow: no dump to take and no maintenance window. Its joining is a request the leader approves - and, with the secret store on, the operator too.

The same request model covers pausing and resuming: the leader, which knows where quorum stands, can refuse a pause that would take the cluster below it.

Tested: rmc; topo t7 · Deploying a cluster

27 · Agent ready#

> `agent.md` puts the whole platform on one page, every subsystem with an honest status: an AI agent onboards as simply as an engineer does, and builds on nothing that is only configured.

agent.md is the whole platform on one page, written for an AI agent - or a new engineer - that has to act on it: every subsystem with its status in the current version (shipping, built, off by default, fixed but not yet green, or not built), the settings that matter, and the conventions to follow, such as which error to retry and which never to.

An agent onboards from one file as simply as an engineer does, and builds on nothing that is only configured.

agent.md

28 · Non-blocking concurrent document operations#

> Two writers editing different paths of one `jsonb` document both land, on every node, and neither waits on the other. The changes commute - `Base + A + B = Base + B + A` - so they apply in any order; edits to the same path are refused, never silently overwritten. Same model as our open-source diff-doc-rs.

The receiving node works out the changed paths from the old and new document every update already carries, and applies only those, each with a precondition that it still holds what the writer read. Edits to different paths compose in either order on every node, and the younger of two writes never waits on the older.

Edits to the same path are refused: with `40001` while the other is in flight, which the driver retries, or with `55409` when it already committed unseen - that one needs a look, not a loop. Arrays merge by position, numbers can merge as increments, plain-text columns by line. Built and in testing: every nightly run exercises it.

Tested: topology 96 and 96b · Two writers, one JSON row, no waiting

29 · Leave any time - a full rollback#

> Sign a node out by deleting its roster row, and every member retires it. To leave altogether, `ytctl --purge` stops the service and removes the extension's files and the cluster WAL; `DROP EXTENSION` takes the extension's tables and triggers with it. Your data never moves: each node is plain PostgreSQL again - no vendor lock, for real.

Signing a node out is deleting its row in the roster: every member retires it. Leaving altogether is `ytctl --purge`, which stops the service and

removes the extension's files and the cluster log, and `DROP EXTENSION … CASCADE`, which takes the extension's tables and triggers with it.

Your data never moved, so there is nothing to migrate back: each node is plain PostgreSQL again, with every table in place. The way out is part of the public package tests, not a promise.

Tested: lxc (purge) · The way out, tested

30 · A Rust server - no JVM#

> `ytserv`, the daemon on every node, is written in Rust: fast, memory safe and small, with no JVM to install, tune or pause for garbage collection. So are `ytctl`, the etcd server, rppd and Schema Guard; the PostgreSQL extension is a small C library.

`ytserv`, one daemon per PostgreSQL instance, is written in Rust: memory safe without a garbage collector, fast and small, with no JVM to install, size or pause. `ytctl`, the etcd server, rppd and Schema Guard are Rust too.

The regular `ytserv` has no unsafe code in its own crates - the compiler refuses it - and what it links is checked against the RustSec advisory database; both at every nightly run that publishes. `ytserv-py`, the build with Python server functions, holds unsafe code: pyo3, the distribution's libpython and one call of its own.

The PostgreSQL extension is a small C library on the write path; the heavier work happens in the daemon, outside the database's own processes.

Main components

you@example.com Subscribe



All trademarks are the property of their respective owners.

© 2020-26 DBinvent